



H.A.M.M.E.R. POS — Guía de Despliegue en Producción

Índice

- 1. [Requisitos previos](#)
- 2. [Arquitectura de producción](#)
- 3. [Configuración de DNS](#)
- 4. [Preparar el servidor](#)
- 5. [Configurar variables de entorno](#)
- 6. [Despliegue paso a paso](#)
- 7. [Seed inicial \(primer despliegue\)](#)
- 8. [Verificación post-despliegue](#)
- 9. [Actualización de la aplicación](#)
- 10. [Backup y restauración](#)
- 11. [Mantenimiento](#)
- 12. [Troubleshooting](#)
- 13. [Despliegue alternativo: Railway](#)

1. Requisitos previos

Servidor (VPS / VM)

Recurso	Mínimo	Recomendado
CPU	1 vCPU	2 vCPUs
RAM	1 GB	2 GB
Disco	20 GB SSD	40 GB SSD
SO	Ubuntu 22.04+ / Debian 12+	Ubuntu 24.04 LTS

Software requerido

- **Docker Engine** ≥ 24.0
- **Docker Compose** ≥ 2.20 (integrado en Docker Engine moderno)
- **Git** (para clonar el repositorio)

Instalación rápida de Docker (Ubuntu)

```
# Instalar Docker Engine
curl -fsSL https://get.docker.com | sh

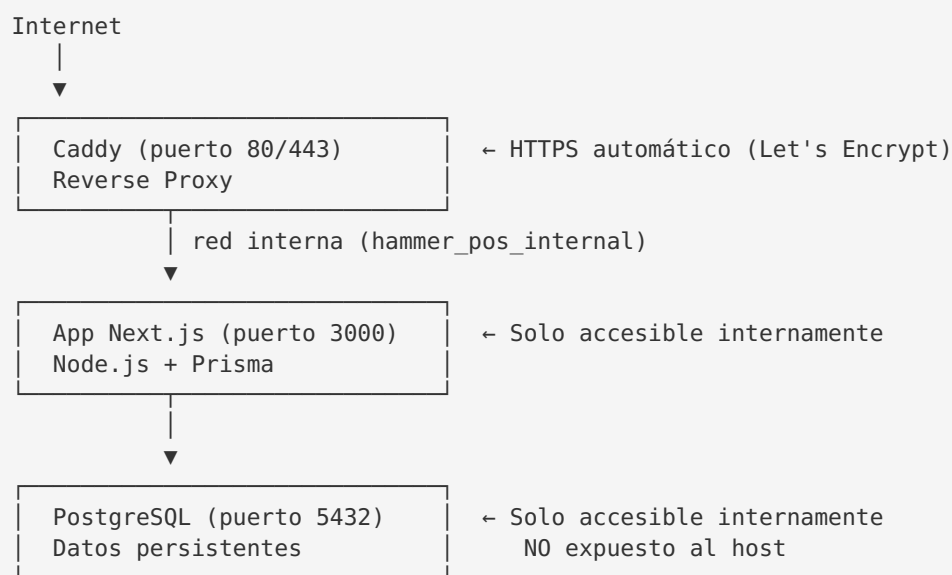
# Agregar tu usuario al grupo docker (evita usar sudo)
sudo usermod -aG docker $USER
newgrp docker

# Verificar instalación
docker --version
docker compose version
```

Dominio y DNS

- Un dominio o subdominio apuntando al servidor (ej: `pos.tuempresa.com`)
- Los puertos **80** y **443** abiertos en el firewall

2. Arquitectura de producción



Características de seguridad:

- PostgreSQL **NO** expone el puerto 5432 al host
- La app Next.js **NO** expone puertos al host
- Solo Caddy expone los puertos 80 (HTTP → redirige a HTTPS) y 443 (HTTPS)
- Todos los servicios se comunican via red interna Docker
- HTTPS automático con certificados de Let's Encrypt
- Headers de seguridad configurados en Caddy (HSTS, X-Frame-Options, etc.)

3. Configuración de DNS

Paso 1: Obtener la IP de tu servidor

```
curl ifconfig.me
```

Paso 2: Crear registro DNS

En tu proveedor de DNS (Cloudflare, Route53, Namecheap, etc.):

Tipo	Nombre	Valor	TTL
A	pos.tuempresa.com	TU_IP_SERVIDOR	300

 **Importante:** El DNS debe estar propagado **antes** de iniciar Caddy, para que Let's Encrypt pueda verificar el dominio y emitir el certificado SSL.

Verificar propagación DNS

```
# Desde cualquier máquina
dig +short pos.tuempresa.com
# Debe retornar la IP de tu servidor

# O usando nslookup
nslookup pos.tuempresa.com
```

Si usas Cloudflare

- Configurar el registro como **DNS only** (nube gris) inicialmente
- Caddy maneja los certificados SSL directamente
- Una vez funcionando, puedes activar proxy de Cloudflare si lo deseas

4. Preparar el servidor

```
# Conectar al servidor
ssh usuario@tu-servidor

# Crear directorio para el proyecto
mkdir -p /opt/hammer-pos
cd /opt/hammer-pos

# Clonar el repositorio
git clone https://github.com/tu-org/hammer-pos.git .
# O copiar los archivos necesarios al servidor
```

Firewall (UFW)

```
sudo ufw allow 22/tcp      # SSH
sudo ufw allow 80/tcp      # HTTP (redirección a HTTPS)
sudo ufw allow 443/tcp     # HTTPS
sudo ufw allow 443/udp     # HTTP/3 (QUIC) – opcional
sudo ufw enable
sudo ufw status
```

5. Configurar variables de entorno

5.1 Archivos de entorno del proyecto

Archivo	Propósito	¿Commitear?
<code>.env.production.example</code>	Plantilla con placeholders para producción	✓ Sí
<code>.env.production</code>	Valores REALES de producción	✗ NUNCA
<code>.env.example</code>	Plantilla para desarrollo / demo	✓ Sí
<code>.env.local.example</code>	Plantilla para desarrollo local	✓ Sí
<code>.env</code>	Valores para desarrollo local	✗ No

⚠ `.env.production` está en `.gitignore` y **nunca** debe subirse al repositorio.

5.2 Crear el archivo de producción

```
# Copiar plantilla
cp .env.production.example .env.production
```

5.3 Generar secretos seguros

```
# Contraseña para PostgreSQL (POSTGRES_PASSWORD)
openssl rand -base64 32

# Secreto de sesión (AUTH_SESSION_SECRET) – mínimo 32 caracteres
openssl rand -hex 32
# Alternativa con Node.js:
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# Alternativa con Python:
python3 -c "import secrets; print(secrets.token_hex(32))"

# Contraseñas para bootstrap (BOOTSTRAP_*_PASSWORD)
openssl rand -base64 16
```

5.4 Variables obligatorias (DEBEN cambiarse)

Variable	Descripción	Cómo generar
DOMAIN	Dominio real (ej: <code>pos.tuempresa.com</code>)	Configurar en DNS
POSTGRES_PASSWORD	Contraseña de la base de datos	<code>openssl rand -base64 32</code>
AUTH_SESSION_SECRET	Secreto para firmar sesiones (≥ 32 chars)	<code>openssl rand -hex 32</code>
APP_ENV	Debe ser <code>production</code>	Valor fijo
NODE_ENV	Debe ser <code>production</code>	Valor fijo
BOOTSTRAP_OWNER_*	Email, nombre y contraseña del owner	Datos reales + pwd seguro
BOOTSTRAP_SYSADMIN_*	Email, nombre y contraseña del sysadmin	Datos reales + pwd seguro

5.5 Variables opcionales

Variable	Default	Descripción
AUTH_SESSION_TTL_HOURS	12	Duración de sesión en horas (8-12 recomendado)
RUN_MIGRATIONS	false	<code>true</code> para auto-migrar al iniciar el contenedor
ENABLE_CASH_CLOSURE_SCHEDULER	false	Activar cierre automático de caja
PORT	3000	Puerto interno del contenedor
BOOTSTRAP_BRANCH_*	—	Código y nombre de la sucursal inicial
BOOTSTRAP_CREATE_CASH_BOX	true	Crear caja automáticamente con la sucursal

Sobre RUN_MIGRATIONS : Usar `true` para despliegues simples (Docker Compose directo). Usar `false` si prefieres controlar las migraciones en un pipeline CI/CD separado.

5.6 Editar y guardar

```
nano .env.production
```

5.7 Validar la configuración

El proyecto incluye un script de validación que verifica:

- Que `AUTH_SESSION_SECRET` no sea un placeholder ni valor inseguro (mínimo 32 chars)
- Que `DATABASE_URL` sea una URL PostgreSQL válida
- Que `APP_ENV` y `NODE_ENV` sean `production` en modo estricto
- Advertencias sobre credenciales por defecto

```
# Validar manualmente (modo estricto para producción)
APP_ENV=production NODE_ENV=production node scripts/validate-env.mjs --mode=strict

# O usando npm (se ejecuta automáticamente con npm run build y npm run dev)
npm run env:validate
```

El script `validate-env.mjs` se ejecuta automáticamente antes de `npm run dev`, `npm run build` y `npm run start` (en modo estricto). Si detecta valores inseguros, **abortará la ejecución**.

6. Despliegue paso a paso

6.1 Construir y levantar servicios

```
cd /opt/hammer-pos

# Construir la imagen de la aplicación
docker compose -f docker-compose.production.yml --env-file .env.production build

# Levantar todos los servicios en background
docker compose -f docker-compose.production.yml --env-file .env.production up -d
```

6.2 Verificar que los servicios están corriendo

```
# Ver estado de los servicios
docker compose -f docker-compose.production.yml ps

# Todos deben estar en estado "Up" o "healthy"
```

6.3 Ver logs en tiempo real

```
# Todos los servicios
docker compose -f docker-compose.production.yml logs -f

# Solo la aplicación
docker compose -f docker-compose.production.yml logs -f app

# Solo Caddy
docker compose -f docker-compose.production.yml logs -f caddy

# Solo PostgreSQL
docker compose -f docker-compose.production.yml logs -f db
```

7. Seed inicial (primer despliegue)

En el **primer despliegue**, ejecutar el seed de producción para crear usuarios y sucursal:

```
docker compose -f docker-compose.production.yml --env-file .env.production \
  exec app npm run seed:production
```

Nota: Solo necesario una vez. Las variables `BOOTSTRAP_*` en `.env.production` definen las credenciales del owner y admin inicial.

8. Verificación post-despliegue

Checks automáticos

```
# Health check de la app
curl -s https://pos.tuempresa.com/health | jq .

# Verificar certificado SSL
curl -vI https://pos.tuempresa.com 2>&1 | grep -i "SSL\|certificate\|subject"

# Verificar headers de seguridad
curl -sI https://pos.tuempresa.com | grep -iE "strict-transport|x-frame|x-content-type"
```

Checks manuales

1. ☒ Abrir `https://pos.tuempresa.com` en el navegador
2. ☒ Verificar que muestra el candado SSL (HTTPS)
3. ☒ Login con credenciales del bootstrap
4. ☒ Navegar al dashboard
5. ☒ Probar apertura y cierre de caja
6. ☒ Realizar una venta de prueba

9. Actualización de la aplicación

```
cd /opt/hammer-pos

# Obtener últimos cambios
git pull origin main

# Reconstruir solo la app (sin downtime en DB)
docker compose -f docker-compose.production.yml --env-file .env.production build app

# Reiniciar con los cambios
docker compose -f docker-compose.production.yml --env-file .env.production up -d

# Verificar que todo funciona
docker compose -f docker-compose.production.yml ps
docker compose -f docker-compose.production.yml logs -f app --tail=50
```

Actualización con zero-downtime (opcional)

```
# Reconstruir imagen
docker compose -f docker-compose.production.yml --env-file .env.production build app

# Reiniciar solo el servicio app
docker compose -f docker-compose.production.yml --env-file .env.production up -d --no-deps app

# Caddy sigue sirviendo mientras la app se reinicia
```

10. Backup y restauración

Backup de PostgreSQL

```
# Backup completo (SQL dump)
docker compose -f docker-compose.production.yml exec db \
  pg_dump -U hammer -d hammer_pos --format=custom \
  > backup_$(date +%Y%m%d_%H%M%S).dump

# Backup solo datos (sin schema)
docker compose -f docker-compose.production.yml exec db \
  pg_dump -U hammer -d hammer_pos --data-only \
  > backup_data_$(date +%Y%m%d_%H%M%S).sql
```

Restaurar backup

```
# Restaurar desde dump custom
docker compose -f docker-compose.production.yml exec -i db \
  pg_restore -U hammer -d hammer_pos --clean --if-exists \
  < backup_20250512_120000.dump
```


Script de backup automático (cron)

```
# Crear script de backup
cat > /opt/hammer-pos/scripts/backup.sh << 'EOF'
#!/bin/bash
BACKUP_DIR="/opt/hammer-pos/backups"
mkdir -p "$BACKUP_DIR"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

cd /opt/hammer-pos
docker compose -f docker-compose.production.yml exec -T db \
  pg_dump -U hammer -d hammer_pos --format=custom \
  > "$BACKUP_DIR/hammer_pos_${TIMESTAMP}.dump"

# Eliminar backups mayores a 30 días
find "$BACKUP_DIR" -name "*.dump" -mtime +30 -delete

echo "[$(date)] Backup completado: hammer_pos_${TIMESTAMP}.dump"
EOF

chmod +x /opt/hammer-pos/scripts/backup.sh

# Agregar al cron (diario a las 2:00 AM)
(crontab -l 2>/dev/null; echo "0 2 * * * /opt/hammer-pos/scripts/backup.sh >> /var/
log/hammer-backup.log 2>&1") | crontab -
```

11. Mantenimiento

Comandos útiles

```
# Alias sugerido (agregar a ~/.bashrc)
alias hpos="docker compose -f /opt/hammer-pos/docker-compose.production.yml --env-
file /opt/hammer-pos/.env.production"

# Con el alias:
hpos ps                # Estado de servicios
hpos logs -f app       # Logs de la app
hpos restart app       # Reiniciar app
hpos down              # Detener todo
hpos up -d             # Levantar todo
```

Monitoreo de recursos

```
# Uso de recursos por contenedor
docker stats --no-stream

# Espacio en disco de volúmenes
docker system df -v
```

Limpieza de Docker

```
# Eliminar imágenes no usadas (liberar espacio)
docker image prune -f

# Limpieza completa (cuidado: elimina todo lo no usado)
docker system prune -f
```

Renovación de certificados SSL

Caddy renueva los certificados de Let's Encrypt **automáticamente** antes de que expiren. No se requiere intervención manual. Los certificados se almacenan en el volumen `caddy_data`.

12. Troubleshooting

✗ Caddy no obtiene certificado SSL

Síntomas: Error "ACME challenge failed" en logs de Caddy.

Solución:

1. Verificar que el DNS apunta correctamente al servidor:

```
bash
dig +short pos.tuempresa.com
```

2. Verificar que los puertos 80 y 443 están abiertos:

```
bash
sudo ufw status
sudo ss -tlnp | grep -E ':80|:443'
```

3. Verificar que no hay otro servicio usando los puertos 80/443:

```
bash
sudo lsof -i :80
sudo lsof -i :443
```

4. Reiniciar Caddy:

```
bash
docker compose -f docker-compose.production.yml restart caddy
```

✗ La app no conecta a PostgreSQL

Síntomas: "Connection refused" o "ECONNREFUSED" en logs de app.

Solución:

1. Verificar que PostgreSQL está healthy:

```
bash
docker compose -f docker-compose.production.yml ps db
docker compose -f docker-compose.production.yml logs db --tail=20
```

2. Verificar que `DATABASE_URL` es correcto (user/pass/host/db coinciden con variables `POSTGRES_*`):

```
bash
docker compose -f docker-compose.production.yml exec app env | grep DATABASE_URL
```

3. Probar conexión desde el contenedor de la app:

```
bash
docker compose -f docker-compose.production.yml exec app \
sh -c "wget -qO- http://localhost:3000/health"
```

✗ Error 502 Bad Gateway

Síntomas: Caddy retorna 502.

Solución:

1. La app probablemente no está lista. Verificar:

```
bash
```

```
docker compose -f docker-compose.production.yml logs -f app --tail=50
```

2. Esperar a que el healthcheck de la app pase (puede tardar ~60s en el primer inicio).

3. Si persiste, verificar que la app escucha en el puerto 3000:

```
bash
```

```
docker compose -f docker-compose.production.yml exec app \
  sh -c "wget -qO- http://localhost:3000/health"
```

✗ Pantalla en blanco / Error de JS

Solución:

1. Verificar logs de la app para errores de build/runtime

2. Ejecutar validación de entorno:

```
bash
```

```
docker compose -f docker-compose.production.yml exec app npm run env:validate
```

3. Verificar que las migraciones se aplicaron:

```
bash
```

```
docker compose -f docker-compose.production.yml exec app npx prisma migrate status
```

✗ Migraciones fallan al iniciar

Solución:

1. Verificar estado de migraciones:

```
bash
```

```
docker compose -f docker-compose.production.yml exec app npx prisma migrate status
```

2. Si hay migraciones fallidas, revisar y resolver manualmente:

```
bash
```

```
docker compose -f docker-compose.production.yml exec app npx prisma migrate resolve --
applied MIGRATION_NAME
```

✗ Sin espacio en disco

```
# Ver uso de disco
```

```
df -h
```

```
# Limpiar imágenes Docker antiguas
```

```
docker image prune -a -f
```

```
# Limpiar logs de contenedores
```

```
docker compose -f docker-compose.production.yml logs --tail=0
truncate -s 0 /var/lib/docker/containers/*/json.log
```

13. Despliegue alternativo: Railway

Para entornos de staging o si prefieres PaaS en vez de VPS:

Variables requeridas en Railway

- `DATABASE_URL` — referencia al servicio PostgreSQL de Railway
- `AUTH_SESSION_SECRET` — 32+ caracteres aleatorios
- `AUTH_SESSION_TTL_HOURS` — ej: 12
- `NODE_ENV=production`
- `APP_ENV=production`

Build / migrate / start

- **Build:** `npm run build` (ejecuta `prisma generate` + `next build`)
- **Pre-deploy:** `npm run railway:migrate` (espera DB + `prisma migrate deploy`)
- **Start:** `npm run start:railway` (host `0.0.0.0`, puerto `${PORT}`)

Migraciones

```
npm run prisma:validate
npm run prisma:generate
npm run prisma:migrate:deploy
```

Seed

- Demo/Staging: `npm run seed:demo`
- Productivo mínimo: `npm run seed:production`

Checklist de despliegue

- ☐ DNS configurado y propagado
- ☐ Puertos 80 y 443 abiertos en firewall
- ☐ `.env.production` con valores reales (no placeholders)
- ☐ `POSTGRES_PASSWORD` es una contraseña fuerte
- ☐ `AUTH_SESSION_SECRET` tiene 32+ caracteres
- ☐ `DOMAIN` configurado con tu dominio real
- ☐ Servicios levantados: `docker compose ... up -d`
- ☐ Health check pasa: `curl https://tu-dominio/health`
- ☐ Certificado SSL válido (candado verde en navegador)
- ☐ Seed de producción ejecutado (primer despliegue)
- ☐ Login exitoso con credenciales bootstrap
- ☐ Backup automático configurado (cron)